

Guide Conceptuel : Architecture Backend Moderne*

**Qu'importe le langage choisi (Java, PHP, Python, Node.js, etc.).*





Architecture : Définition Concrète dans le Développement





Qu'est-ce que l'architecture concrètement ?

ARCHITECTURE = 3 CHOSES

 1. ORGANISATION DES FICHIERS
Comment vous structurez votre projet

 2. RELATIONS ENTRE LES COMPOSANTS
Qui appelle qui, dans quel sens

 3. RÈGLES ET CONTRAINTES
Ce qui est autorisé ou interdit





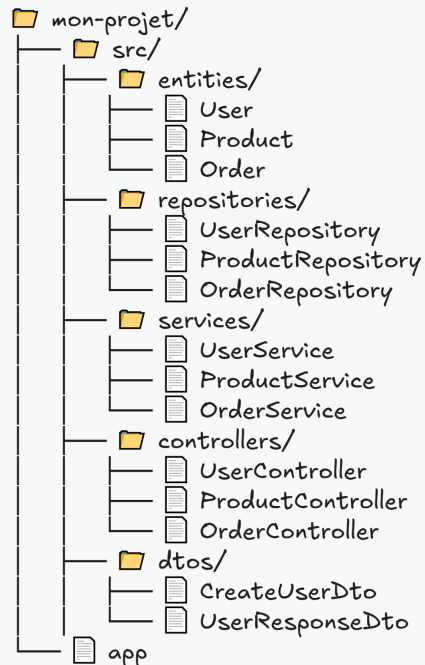
1. ORGANISATION DES FICHIERS

✗ MAUVAISE Architecture (Tout mélangé)



(Controller + Service + Database)
(Tout sur les users)
(Tout sur les produits)
(Tout sur les commandes)
(Configuration DB)

✓ BONNE Architecture (Séparation claire)





2. RELATIONS ENTRE COMPOSANTS

🎯 Règle Fondamentale : Le Sens des Dépendances

FLUX DES APPELS



CLIENT

↓ HTTP Request



CONTROLLER

→ importe et utilise →



SERVICE



importe et utilise



REPOSITORY



DATABASE



AUTORISÉ : Controller → Service → Repository



INTERDIT : Repository → Service → Controller





3. RÈGLES ET CONTRAINTES

🎯 Les Règles Concrètes de l'Architecture

RÈGLES STRICTES

🚫 INTERDICTIONS :

- ❌ Contrôleur ne peut PAS importer Repository
// ❌ INTERDIT
- ❌ Repository ne peut PAS contenir de logique métier
// ❌ INTERDIT dans Repository
if (user.age < 18) throw new Error('Trop jeune');
- ❌ Entité ne peut PAS être exposée directement
// ❌ INTERDIT dans Contrôleur
res.json(user); // avec password !

✅ AUTORISATIONS :

- ✅ Contrôleur peut importer Service
- ✅ Service peut importer Repository
- ✅ Service contient TOUTE la logique métier
if (user.age < 18) throw new Error('Trop jeune');

🎯 Pourquoi Ces Règles ?

MÉTAPHORE RESTAURANT

🌐 CONTROLLER = SERVEUR

- Prend la commande du client
- Ne cuisine PAS
- Ne va PAS aux courses
- Transmet au chef et sert le plat

⚙️ SERVICE = CHEF CUISINIER

- Décide des recettes (logique métier)
- Coordonne la cuisine
- Vérifie la qualité
- Ne sert PAS les clients

📖 REPOSITORY = GARDE-MANGER

- Stocke et récupère les ingrédients
- Ne décide PAS des recettes
- Ne cuisine PAS
- Juste l'accès aux ressources

🎯 RÉSULTAT : Restaurant efficace et organisé





Checklist : Votre Architecture est-elle Bonne ?

 Questions à se poser :

CHECKLIST

- ☐ Chaque fichier a-t-il *UN* seul rôle ?
- ☐ Le Controller contient-il *SEULEMENT* du code HTTP ?
- ☐ Le Service contient-il *TOUTE* la logique métier ?
- ☐ Le Repository contient-il *SEULEMENT* des requêtes DB ?
- ☐ Les imports vont-ils dans le bon sens ?
(Controller → Service → Repository)
- ☐ Peut-on tester chaque couche séparément ?
- ☐ Peut-on changer de base de données sans casser le Service ou le Controller ?
- ☐ Les mots de passe ne sont-ils *JAMAIS* exposés ?
- ☐ Les erreurs sont-elles gérées à chaque niveau ?

Si vous répondez *OUI* partout :  **BONNE ARCHITECTURE !**



Architecture avec Interfaces SOLID : Code Professionnel, Évolutif et Testable





Qu'est-ce que l'architecture avec interfaces SOLID ?

ARCHITECTURE = 3 CHOSES

📁 1. ORGANISATION DES FICHIERS
Comment vous structurez votre projet avec interfaces

🕒 2. RELATIONS ENTRE LES COMPOSANTS
Qui appelle qui, dans quel sens (via contrats)

✏️ 3. RÈGLES ET CONTRAINTES
Ce qui est autorisé ou interdit (principes SOLID)





Qu'est-ce que l'architecture avec interfaces SOLID ?

🎯 Qu'apporte l'ajout des Interfaces ?

ARCHITECTURE CLASSIQUE vs AVEC INTERFACES

❌ SANS INTERFACES :

UserService → PostgreSQLUserRepository

- Couplage fort
- Difficile à tester
- Pas d'échange d'implémentation

✅ AVEC INTERFACES :

UserService → IUserRepository ← PostgreSQLUserRepository

- Couplage faible
- Tests avec MockRepository
- PostgreSQL/Mongo/Memory interchangeables



Les 5 Principes SOLID Expliqués Simplement

1. Single Responsibility (S)

UN FICHIER = UN RÔLE

✗ MAUVAIS :

- 📄 UserService
 - Créer users
 - Envoyer emails
 - Gérer la base de données
 - Valider les données

✓ BON :

- 📄 UserService → Logique métier users
- 📄 EmailService → Envoi emails
- 📄 UserRepository → Accès base de données
- 📄 UserValidator → Validation données



Les 5 Principes SOLID Expliqués Simplement

2. Open/Closed (O)

OUVERT À L'EXTENSION,
FERMÉ À LA MODIFICATION



PRINCIPE :

Ajouter des fonctionnalités *SANS* changer
l'existant



EXEMPLE :



IPaymentService (interface)



StripePayment (implémente IPaymentService)



PayPalPayment (implémente IPaymentService)



Pour ajouter Apple Pay :

→ Créer ApplePayPayment

→ Ne PAS toucher aux autres fichiers



Les 5 Principes SOLID Expliqués Simplyment

3. Liskov Substitution (L)

LES IMPLÉMENTATIONS SONT INTERCHANGEABLES



PRINCIPE :

Remplacer PostgreSQL par MongoDB sans casser l'application



EXEMPLE :



IUserRepository

- save(user)
- findById(id)
- findByEmail(email)



PostgreSQLUserRepository ← Implémente IUserRepository



MongoUserRepository ← Implémente IUserRepository



MemoryUserRepository ← Implémente IUserRepository



Le Service ne sait PAS laquelle est utilisée



Les 5 Principes SOLID Expliqués Simplement

4. Interface Segregation (I)

INTERFACES SPÉCIALISÉES

✗ MAUVAIS - Interface trop grosse :

- 📄 IUserService
 - createUser()
 - deleteUser()
 - sendEmail()
 - generateReport()
 - processPayment()

✓ BON - Interfaces spécialisées :

- 📄 IUserService → createUser(), deleteUser()
- 📄 IEmailService → sendEmail()
- 📄 IReportService → generateReport()
- 📄 IPaymentService → processPayment()

💡 Chaque interface a un rôle précis



Les 5 Principes SOLID Expliqués Simplement

5. Dependency Inversion (D)

DÉPENDRE DES ABSTRACTIONS,
PAS DU CONCRET

❌ MAUVAIS :

UserService → PostgreSQLRepository (dépendance directe)

✅ BON :

UserService → IRepository ← PostgreSQLRepository

🎯 AVANTAGES :

- Tests faciles avec MockRepository
- Changer de DB sans casser le Service
- Découplage total





1. ORGANISATION DES FICHIERS

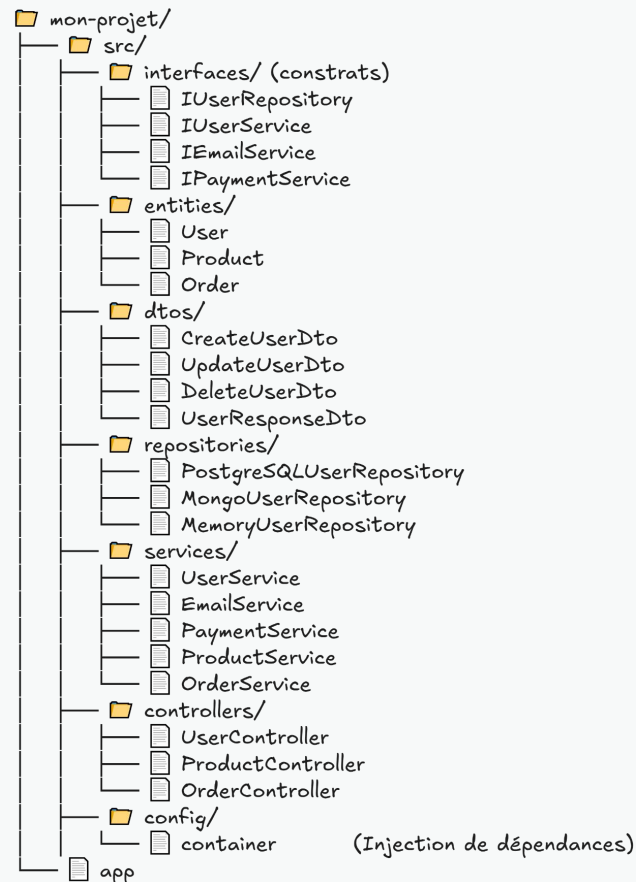


ORGANISATION OPTIMALE

- 📁 interfaces/ (CONTRATS)
 - IUserRepository
 - IUserService
 - IEmailService
- 📁 entities/ (MODÈLES DE DONNÉES)
 - User
 - Product
- 📁 dtos/ (OBJETS DE TRANSFERT)
 - CreateUserDto
 - UserResponseDto
- 📁 repositories/ (IMPLÉMENTATIONS DB)
 - PostgreSQLUserRepository
 - MongoUserRepository
- 📁 services/ (LOGIQUE MÉTIER)
 - UserService.
 - EmailService
- 📁 controllers/ (COUCHE HTTP)
 - UserController
- 📁 config/ (INJECTION DE DÉPENDANCES)
 - container



BONNE Architecture avec Interfaces





2. RELATIONS ENTRE COMPOSANTS

⚡ Flux d'Exécution avec Interfaces

📱 CLIENT

↓ HTTP POST /users



UserController

- Reçoit Request
- Crée CreateUserDto
- Valide les données
- Appelle → IUserService



UserService (implémente IUserService)

- Logique métier
- Hash password
- Appelle → IUserRepository
- Appelle → IEmailService



PostgreSQLUserRepository (implémente IUserRepository)

- Exécute SQL
- Retourne User entity



EmailService (implémente IEmailService)

- Envoie email de bienvenue
- Log des actions



Retour vers le client :

UserResponseDto → JSON → Client





3. RÈGLES ET CONTRAINTES

Interdictions Strictes

RÈGLES STRICTES

INTERDICTIONS :

-  Controller importe Repository directement
-  Service importe implémentation concrète
-  Repository contient logique métier
-  Interface avec trop de méthodes
-  Exposer Entity directement au client
-  Hard-coder les dépendances

AUTORISATIONS :

-  Controller utilise interface IService
-  Service utilise interface IRepository
-  Multiple implémentations d'une interface
-  Injection de dépendances via constructeur
-  Tests avec Mock implementations
-  DTOs pour transfert de données





Qu'est-ce que l'architecture avec interfaces SOLID ?



Avantages de cette Architecture

BÉNÉFICES RÉELS



TESTABILITÉ :

- MockRepository pour les tests unitaires
- Tests isolés par couche



FLEXIBILITÉ :

- Changer PostgreSQL → MongoDB en 5 min
- Ajouter Redis cache sans casser le code



TRAVAIL EN ÉQUIPE :

- Dev A : Interfaces + Services
- Dev B : Repositories
- Dev C : Controllers



ÉVOLUTIVITÉ :

- Nouvelles fonctionnalités sans régression
- Refactoring sécurisé



SÉCURITÉ :

- Séparation claire des responsabilités
- Validation à chaque niveau





Checklist : Votre Architecture est-elle Bonne ?

 Questions à se poser :

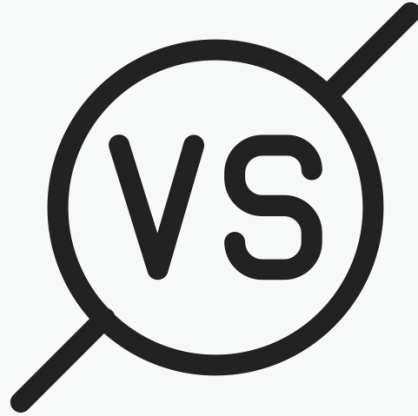
CHECKLIST

- ☐ Dossier interfaces/ existe et contient les contrats ?
- ☐ Services dépendent d'interfaces, pas d'implémentations ?
- ☐ Peut-on changer PostgreSQL → MongoDB en changeant 1 ligne ?
- ☐ Tests unitaires possibles avec Mock ?
- ☐ Chaque interface a moins de 7 méthodes ?
- ☐ Injection de dépendances configurée ?
- ☐ DTOs utilisés pour toutes les entrées/sorties ?
- ☐ Impossible d'accéder DB depuis Controller ?
- ☐ Nouvelles fonctionnalités ajoutables sans régression ?

Si vous répondez OUI partout :  ARCHITECTURE SOLID !



**Architecture
"MVC (Model -
View - Controller)"
Traditionnel
(Monolithique)**



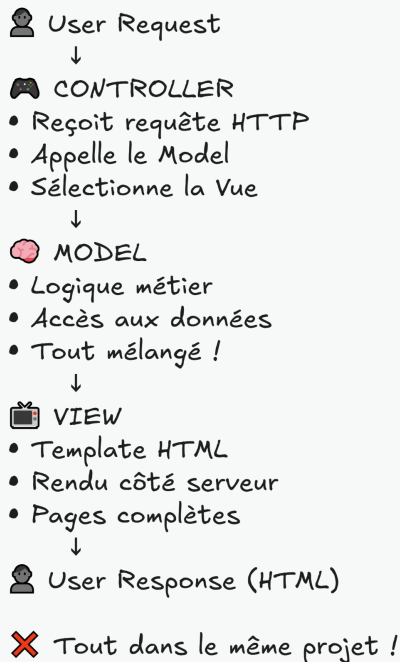
**Architecture
"Clean Layered
Architecture"
Moderne
(en Couches)**



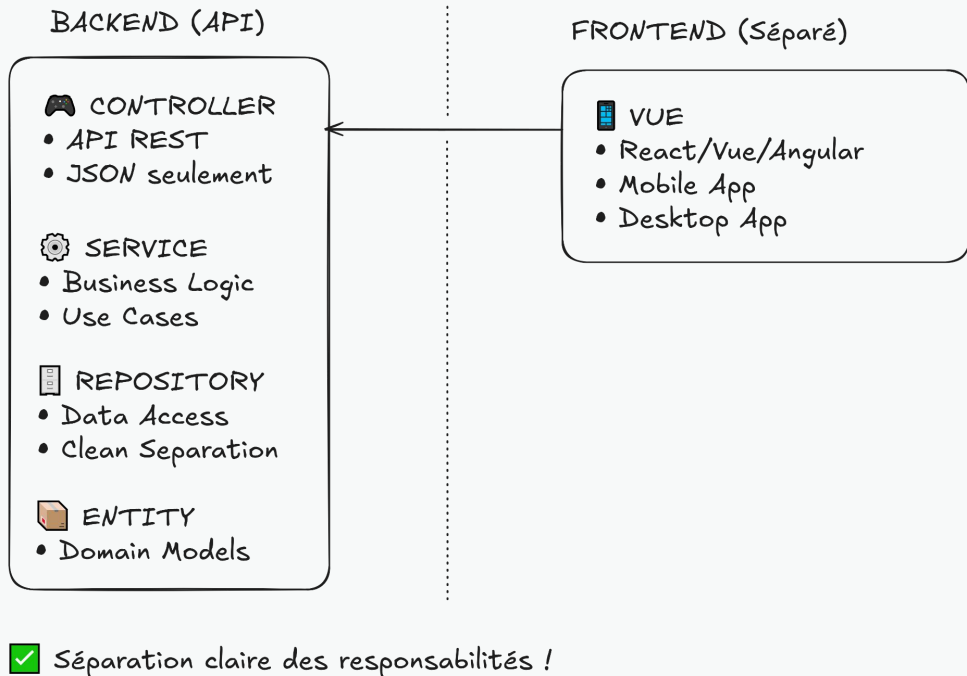


Comparaison MVC Traditionnel (Monolithique) avec Architecture Moderne (en Couches)

MVC TRADITIONNEL (Monolithique)



Architecture Moderne (en Couches)





Comparaison MVC Traditionnel (Monolithique) avec Architecture Moderne (en Couches)

MVC TRADITIONNEL (Monolithique) vs Architecture Moderne (en Couches)		
ASPECT	MVC TRADITIONNEL	ARCHITECTURE MODERNE
Vue	Côté serveur	Côté client
Controller	Gère la vue	API REST seulement
Model	Tout mélangé	Service+Repository
Séparation	Faible	Forte
Testabilité	Difficile	Facile
Scalabilité	Limitée	Excellente
Frontend	Couplé	Découplé
Mobile	Difficile	Natif
API	Secondaire	Prioritaire





Comparaison MVC Traditionnel (Monolithique) avec Architecture Moderne (en Couches)

... Nommage de l'Architecture Moderne :

NOMMAGE OFFICIEL



NOM PRINCIPAL :

"Clean Architecture with Layered Pattern"



NOMS ALTERNATIFS :

- "RESTful API with Clean Architecture"
- "Layered Architecture with Repository Pattern"
- "Service-Repository Pattern with Clean Layers"



POUR LES ÉQUIPES :

"Controller-Service-Repository Architecture"



POUR LA TECH :

"Hexagonal Architecture with REST API"

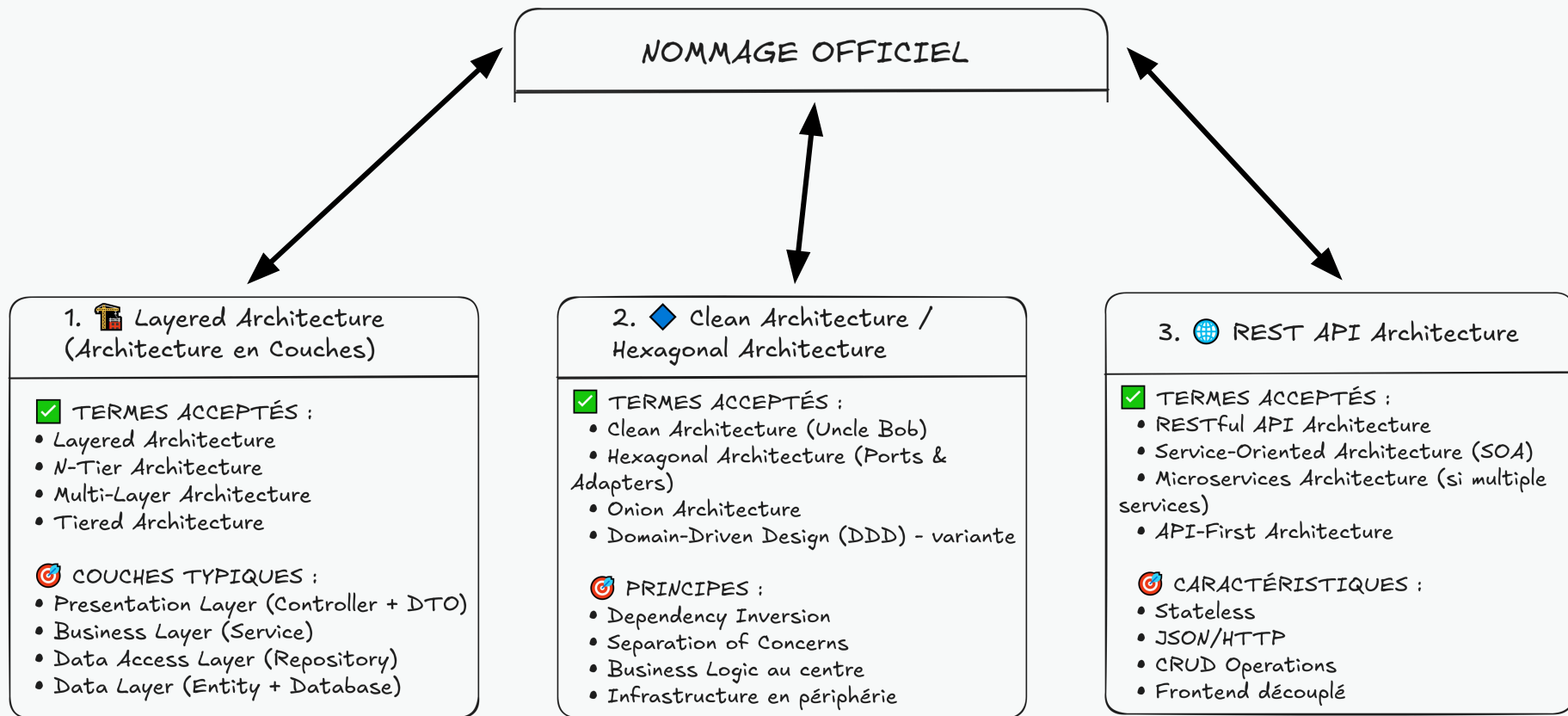


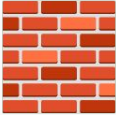
NE DITES PAS :

- "MVC" (c'est faux)
- "Model-View-Controller" (pas de vue backend)

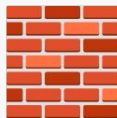


... Nommage de l'Architecture Moderne :



  **Construire un backend
professionnel avec l'architecture la plus
utilisée dans l'industrie !**





Notre Architecture Moderne



ARCHITECTURE EN COUCHES (Layered Architecture)



VOTRE APP/SITE WEB



Communique par Internet (HTTP/JSON)



COUCHE 1 : CONTROLLER

"La réception de votre restaurant"

- Reçoit les demandes des clients
- Renvoie les réponses



Données nettoyées



COUCHE 2 : SERVICE

"Le chef cuisinier"

- Applique les règles de votre business
- Vérifie que tout est correct



Opérations validées



COUCHE 3 : REPOSITORY

"Le garde-manger"

- Récupère et stocke les données
- Parle à la base de données



Requêtes SQL



BASE DE DONNÉES





Les 5 Pièces du Puzzle (1)

1. ENTITY (Le Modèle de vos données)



C'est quoi ?

La structure exacte de vos données



Exemple concret : Un Utilisateur



USER

id (numéro unique)

nom

email

mot_de_passe

date_creation



Règle simple : 1 Entity = 1 table en base de donnée





Les 5 Pièces du Puzzle (2)

2. 📄 DTO (Les Formats d'échange)



C'est quoi ?

Ce qui ENTRE et SORT de votre API



Exemple : Créer un utilisateur



CE QUI RENTRE (CreateUserDTO)

- ✓ nom
- ✓ email
- ✓ mot_de_passe
- ✗ JAMAIS l'id (auto-généré)
- ✗ JAMAIS la date (auto-générée)



CE QUI SORT (UserResponseDTO)

- ✓ id
- ✓ nom
- ✓ email
- ✗ JAMAIS le mot_de_passe (SÉCURITÉ !)
- ✓ date_creation



SÉCURITÉ : Contrôlez exactement ce qui passe !





Les 5 Pièces du Puzzle (3)

3. 📁 REPOSITORY (L'accès aux données)



C'est quoi ?

Le seul endroit qui parle à la base de données



Responsabilités :

- ✓ CRUD (Create, Read, Update, Delete)
- ✓ Requêtes simples
- ✓ Filtres et recherches

- ✗ PAS de logique métier
- ✗ PAS de validation
- ✗ PAS de calculs complexes



Exemple : UserRepository

- saveUser(user) → Sauvegarder
- findUserById(id) → Chercher par ID
- findUserByEmail(email) → Chercher par email
- deleteUser(id) → Supprimer





Les 5 Pièces du Puzzle (4)

4. ⚙️ SERVICE (Le cerveau métier)

🤔 C'est quoi ?

Là où vivent TOUTES vos règles business (logique métier)

🎯 Responsabilités :

- ✓ Valider les données
- ✓ Appliquer les règles métier
- ✓ Gérer la logique complexe
- ✓ Coordonner les opérations
- ✓ Gérer les erreurs métier

📝 Exemple : Créer un utilisateur

- 1 Vérifier que l'email n'existe pas déjà
- 2 Valider le format de l'email
- 3 Vérifier la force du mot de passe
- 4 Hasher le mot de passe
- 5 Appeler le Repository pour sauver
- 6 Envoyer email de bienvenue (optionnel)





Les 5 Pièces du Puzzle (5)

5. 🌐 CONTROLLER (L'interface publique)



C'est quoi ?

Les portes d'entrée de votre API



Responsabilités :

- ✓ Recevoir les requêtes HTTP
- ✓ Valider le format des données
- ✓ Appeler le bon Service
- ✓ Renvoyer la réponse HTTP
- ✓ Gérer les erreurs HTTP
- ✗ PAS de logique métier
- ✗ PAS d'accès direct à la base de donnée



Exemple : Endpoints pour User

- POST /users → Créer
- GET /users/:id → Récupérer un
- GET /users → Lister tous
- PUT /users/:id → Modifier
- DELETE /users/:id → Supprimer





Comment ça s'articule ?

Flux de création d'un utilisateur

CLIENT

Envoie : POST /users

Body: {"nom": "John", "email": "john@test.com",
"mot_de_passe": "123456789"}



CONTROLLER

1 Vérifie le format JSON

2 Convertit en CreateUserDTO

3 Appelle userService.createUser(dto)



SERVICE

1 Valide l'email (format correct ?)

2 Vérifie si l'email existe déjà (via Repository)

3 Valide le mot de passe (assez fort ?)

4 Hashe le mot de passe

5 Crée une Entity User

6 Appelle userRepository.save(user)



REPOSITORY

1 Exécute INSERT INTO users...

2 Retourne l'user avec son ID



SERVICE (retour)

1 Convertit User en UserResponseDTO

2 (Optionnel) Envoie email de bienvenue



CONTROLLER (retour)

1 Retourne HTTP 201 Created

2 Body: {"id": 1, "nom": "John",
"email": "john@test.com", "date": "..."}
↓

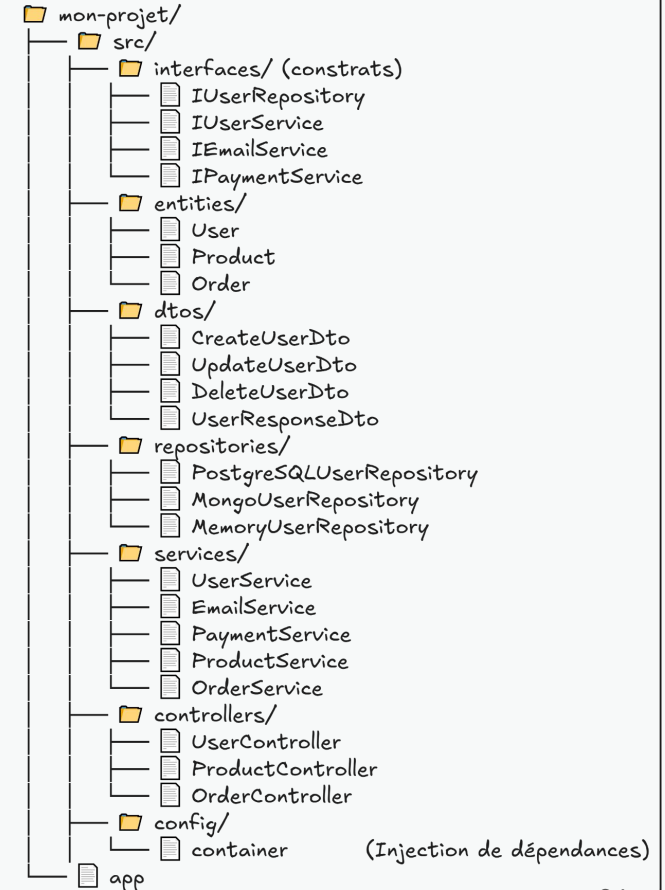
CLIENT

Reçoit la confirmation avec les données de l'user



Comment organiser vos fichiers ?

✓ BONNE Architecture avec Interfaces



Par où commencer ? (Bottom-Up)

ÉTAPES DE CONSTRUCTION (Bottom-Up)

- ☐ ÉTAPE 1 : Créer l'Entity avec interface
Définir la structure de vos données
 Exemple : User avec id, nom, email, password
- ☐ ÉTAPE 2 : Créer les DTOs avec interface
Définir ce qui rentre et sort
 CreateUserDTO, UpdateUserDTO, UserResponseDTO
- ☐ ÉTAPE 3 : Créer le Repository avec interface
Implémenter l'accès aux données
 CRUD de base : save, find, update, delete
- ☐ ÉTAPE 4 : Créer le Service avec interface
Implémenter la logique métier
 Validations, règles business, orchestration
- ☐ ÉTAPE 5 : Créer le Controller avec interface
Exposer les endpoints HTTP
 Routes REST : GET, POST, PUT, DELETE
- ☐ ÉTAPE 6 : Tester votre API
Vérifier que tout fonctionne
 Postman, curl, votre front, ou tests automatisés





Règles d'Or à Retenir

BONNES PRATIQUES

- ✓ UNE responsabilité par classe
- ✓ Controller → Service → Repository → Database
- ✓ Toujours utiliser des DTOs pour l'API
- ✓ Jamais exposer les mots de passe
- ✓ Valider les données à tous les niveaux
- ✓ Gérer les erreurs proprement
- ✓ Nommer clairement (UserService, UserRepository...)
- ✓ Tester chaque couche séparément

ERREURS COURANTES

- ✗ Logique métier dans le Controller
- ✗ Controller qui appelle directement le Repository
- ✗ Repository qui fait de la validation
- ✗ Exposer les Entités directement dans l'API
- ✗ Mélanger les responsabilités
- ✗ Une classe qui fait tout
- ✗ Ignorer la gestion d'erreurs
- ✗ Ne pas valider les entrées





Pourquoi cette architecture ?

Pourquoi cette architecture est top ?



TESTABLE

Chaque couche se teste indépendamment



MAINTENABLE

Changement facile sans tout casser



COLLABORATIVE

Équipe peut travailler en parallèle



ÉVOLUTIVE

Ajouter features sans refactoring massif



SÉCURISÉE

Contrôle total sur les données exposées



STANDARD

Utilisée par toutes les grosses entreprises



PATTERN "Layered Architecture" avec couches SERVICE, Repository et DTO

Ce pattern Model -> Repository -> Service -> Controller (et les DTO que vous choisirez d'y affilier) sera à répéter autant de fois que vous aurez de models différents (produit, order, payment, book): 1 model = 1 répétition de pattern

